

STRINGS IN JAVA

1. StringBuilder – Append

Problem Statement

Write a Java program to demonstrate the use of StringBuilder and its append() method to add text to an existing string.

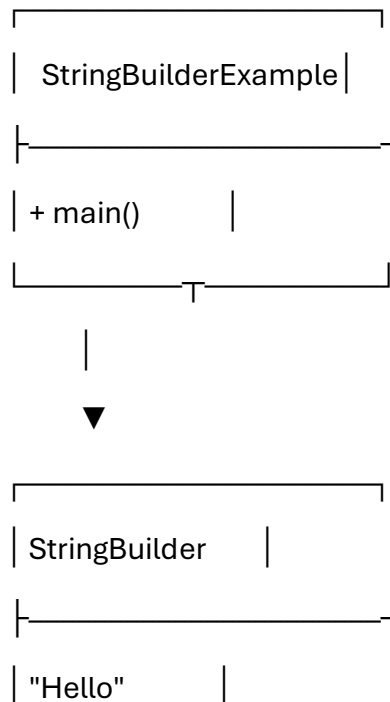
Steps

- Create a StringBuilder object with the string "Hello".
- Use the append() method to add " Java".
- Display the resulting string.

Algorithm

- Start.
- Create a StringBuilder object with "Hello".
- Append " Java" to the object.
- Display the resulting string.
- Stop.

UML Diagram



```
| append(" Java") |
```

Code

```
package strings;

class StringBuilderExample {

    public static void main(String[] args) {

        StringBuilder str = new StringBuilder("Hello");

        str.append(" Java");

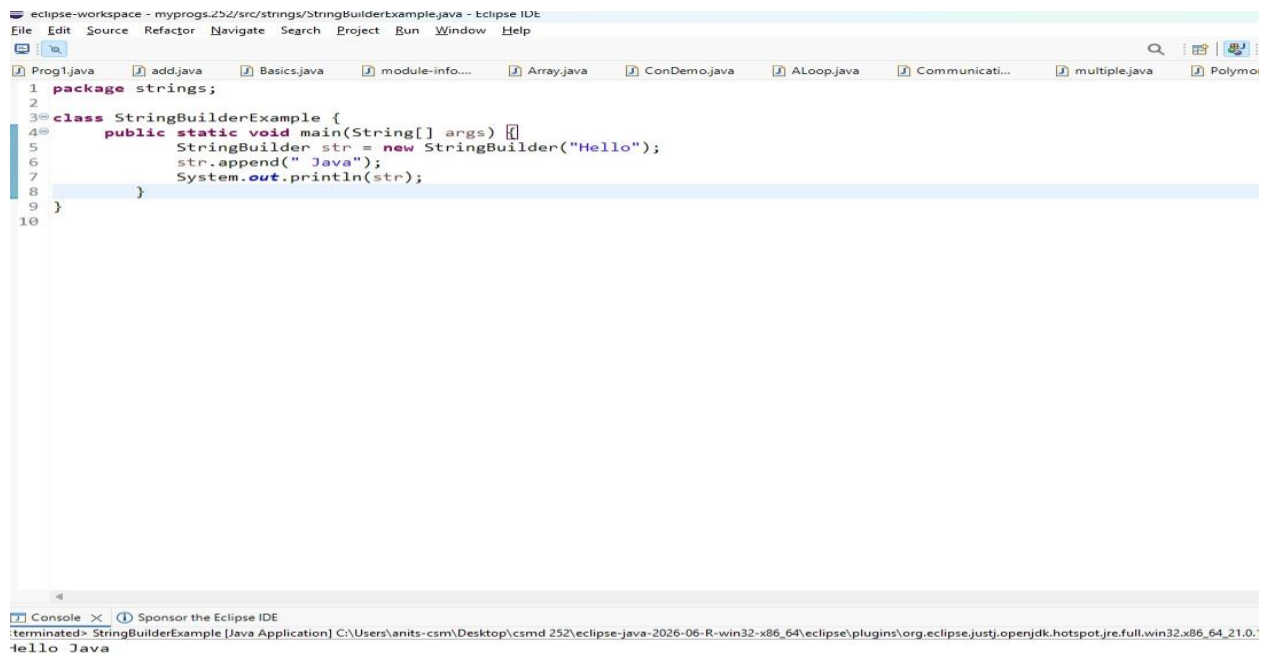
        System.out.println(str);

    }

}
```

Output

Hello Java



2. Byte Array to String

Problem Statement

Write a Java program to demonstrate how a byte array can be converted into a String using the String constructor.

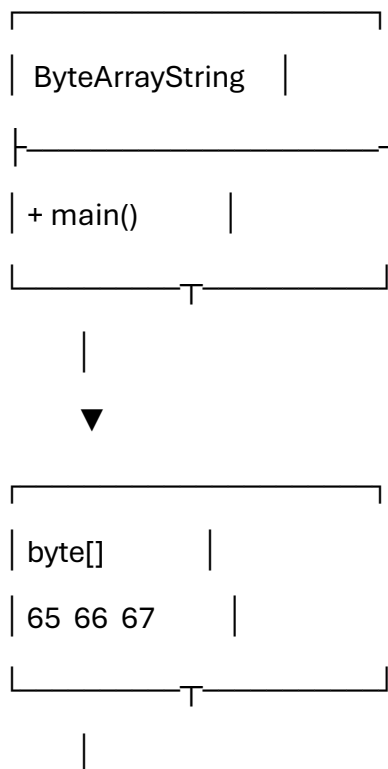
Steps

- Create a byte array containing 65, 66, 67.
- Convert the byte array into a String.
- Display the String.

Algorithm

- Start.
- Create a byte array.
- Store 65, 66, 67 in the array.
- Create a String using the byte array.
- Display the String.
- Stop.

UML Diagram





String	
"ABC"	

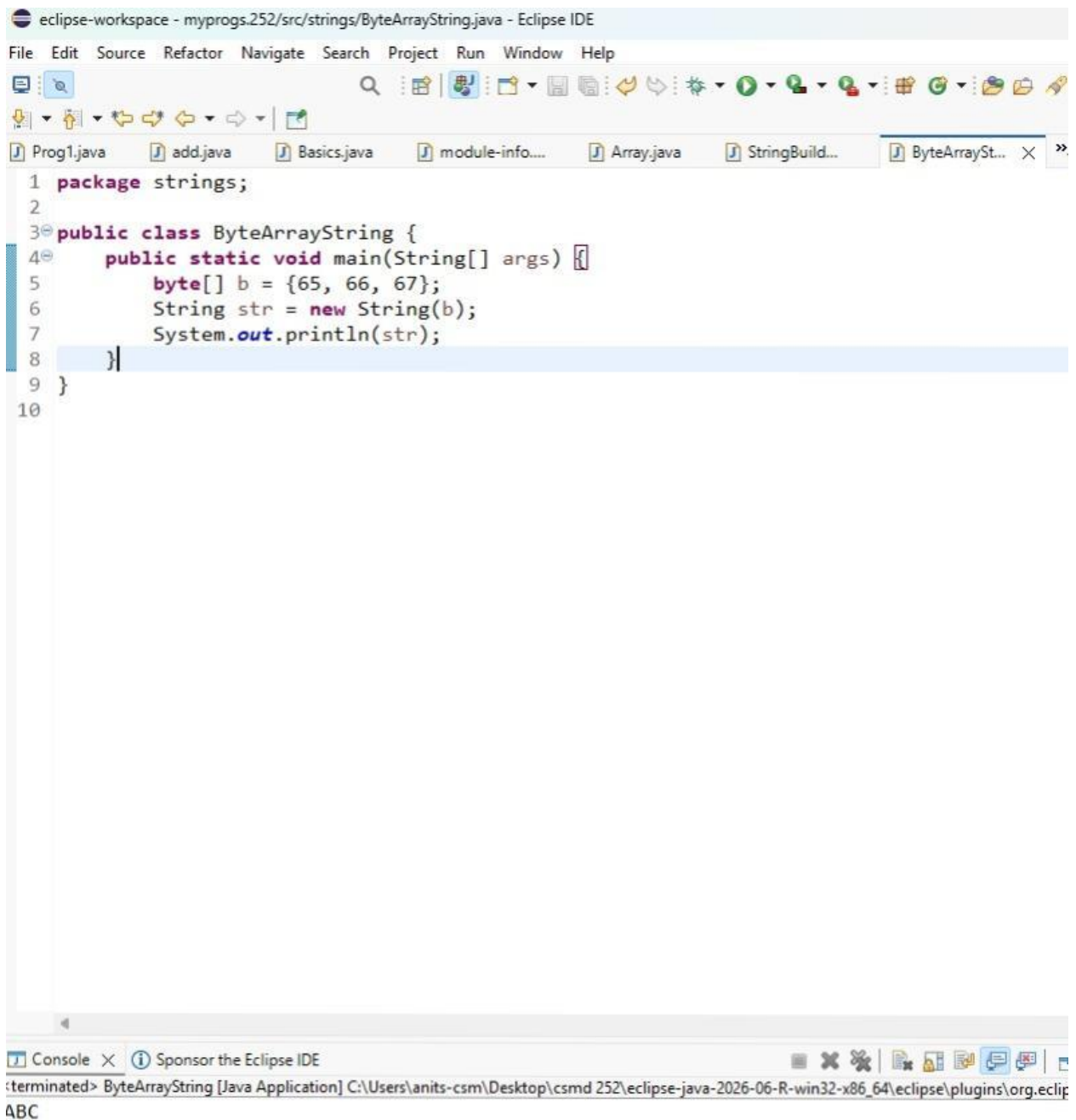
Code

```
package strings;

class ByteArrayString {
    public static void main(String[] args) {
        byte[] b = {65, 66, 67};
        String str = new String(b);
        System.out.println(str);
    }
}
```

Output

ABC



The screenshot shows the Eclipse IDE interface. The title bar reads "eclipse-workspace - myprogs.252/src/strings/ByteArrayString.java - Eclipse IDE". The menu bar includes File, Edit, Source, Refactor, Navigate, Search, Project, Run, Window, and Help. The toolbar contains various icons for file operations and development tools. The package explorer on the left shows a project named "myprogs.252" with a sub-package "strings" containing several Java files: Prog1.java, add.java, Basics.java, module-info..., Array.java, StringBuild..., and ByteArraySt... (selected). The editor window displays the following Java code:

```
1 package strings;
2
3 public class ByteArrayString {
4     public static void main(String[] args) {
5         byte[] b = {65, 66, 67};
6         String str = new String(b);
7         System.out.println(str);
8     }
9 }
10
```

The console at the bottom shows the output of the program:

```
terminated> ByteArrayString [Java Application] C:\Users\anits-csm\Desktop\csm252\eclipse-java-2026-06-R-win32-x86_64\eclipse\plugins\org.eclipse.jdt.launcher\bin\java.exe
ABC
```

3. String charAt()

Problem Statement

Write a Java program to demonstrate the charAt() method to access a character at a particular index in a String.

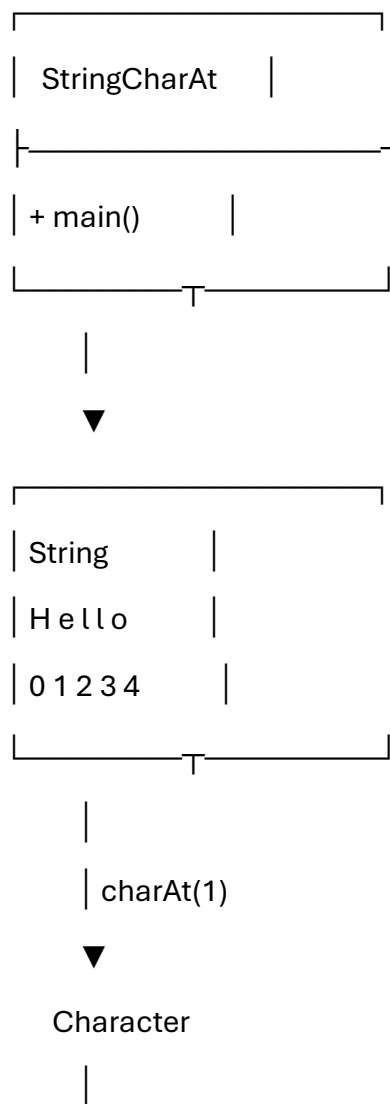
Steps

- Create a String "Hello".
- Use charAt(1) to access the character at index 1.
- Display the character.

Algorithm

- Start.
- Create a String "Hello".
- Call charAt(1).
- Store/display the returned character.
- Stop.

UML Diagram





'e'

Code

```
package strings;

class StringCharAt {

    public static void main(String[] args) {

        String s = "Hello";

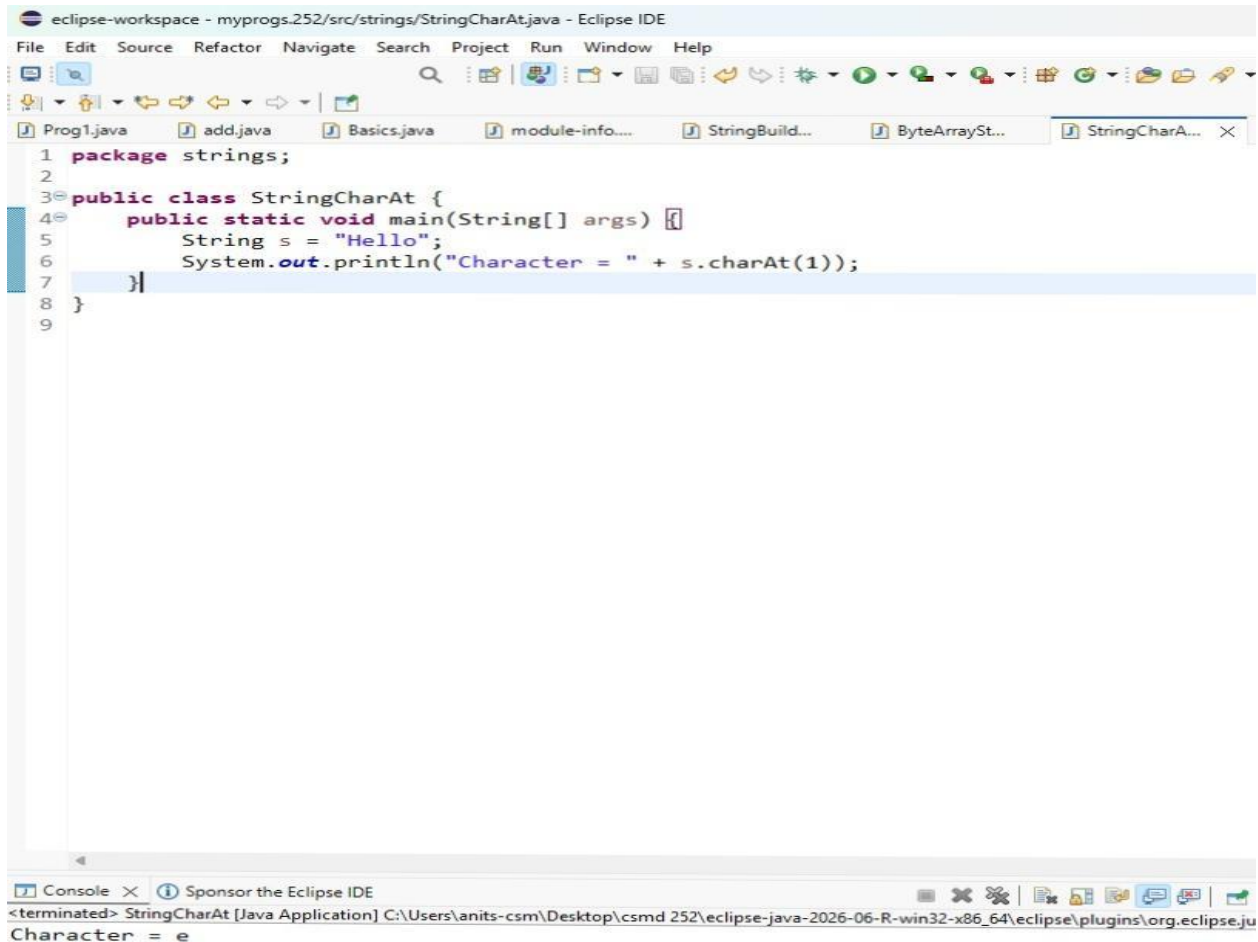
        System.out.println("Character = " + s.charAt(1));

    }

}
```

Output

Character = e



The screenshot shows the Eclipse IDE interface. The title bar reads "eclipse-workspace - myprogs.252/src/strings/StringCharAt.java - Eclipse IDE". The menu bar includes File, Edit, Source, Refactor, Navigate, Search, Project, Run, Window, and Help. The toolbar contains various icons for file operations and development. The package explorer on the left shows a project named "myprogs.252" with a sub-package "strings" containing several Java files: Prog1.java, add.java, Basics.java, module-info..., StringBuild..., ByteArraySt..., and StringCharA... (selected). The editor displays the following Java code:

```
1 package strings;
2
3 public class StringCharAt {
4     public static void main(String[] args) {
5         String s = "Hello";
6         System.out.println("Character = " + s.charAt(1));
7     }
8 }
9
```

The console at the bottom shows the output: "Character = e". The console title is "Console" and it includes a link to "Sponsor the Eclipse IDE". The console text also includes a path: "C:\Users\anits-csm\Desktop\csmd 252\eclipse-java-2026-06-R-win32-x86_64\eclipse\plugins\org.eclipse.ju".

4. Character Array to String

Problem Statement

Write a Java program to demonstrate how a character array can be converted into a String using the String constructor.

Steps

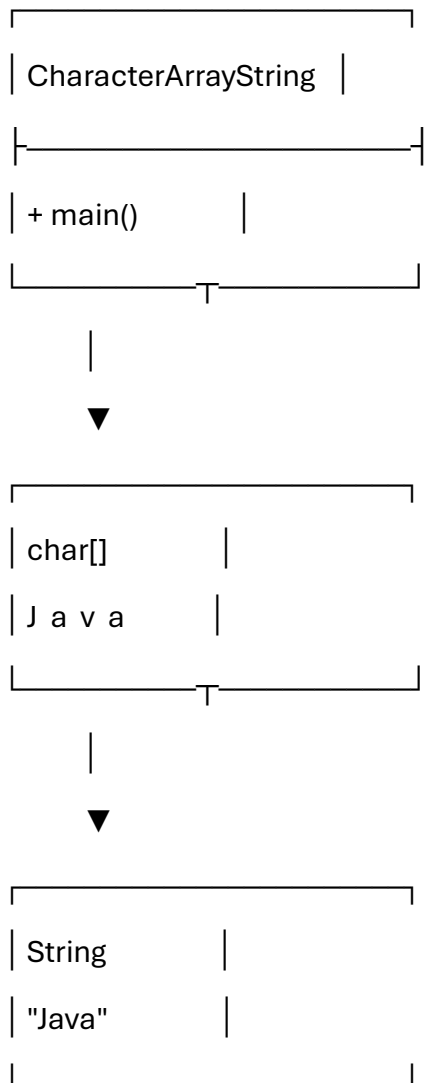
- Create a character array containing J, a, v, a.
- Convert the character array into a String.
- Display the String.

Algorithm

- Start.
- Create a character array.

- Store the characters J, a, v, a.
- Create a String using the character array.
- Display the String.
- Stop.

UML Diagram



Code

```

package strings;

class CharacterArrayString {

    public static void main(String[] args) {

        char[] ch = {'J', 'a', 'v', 'a'};
    }
}
  
```

```
String str = new String(ch);

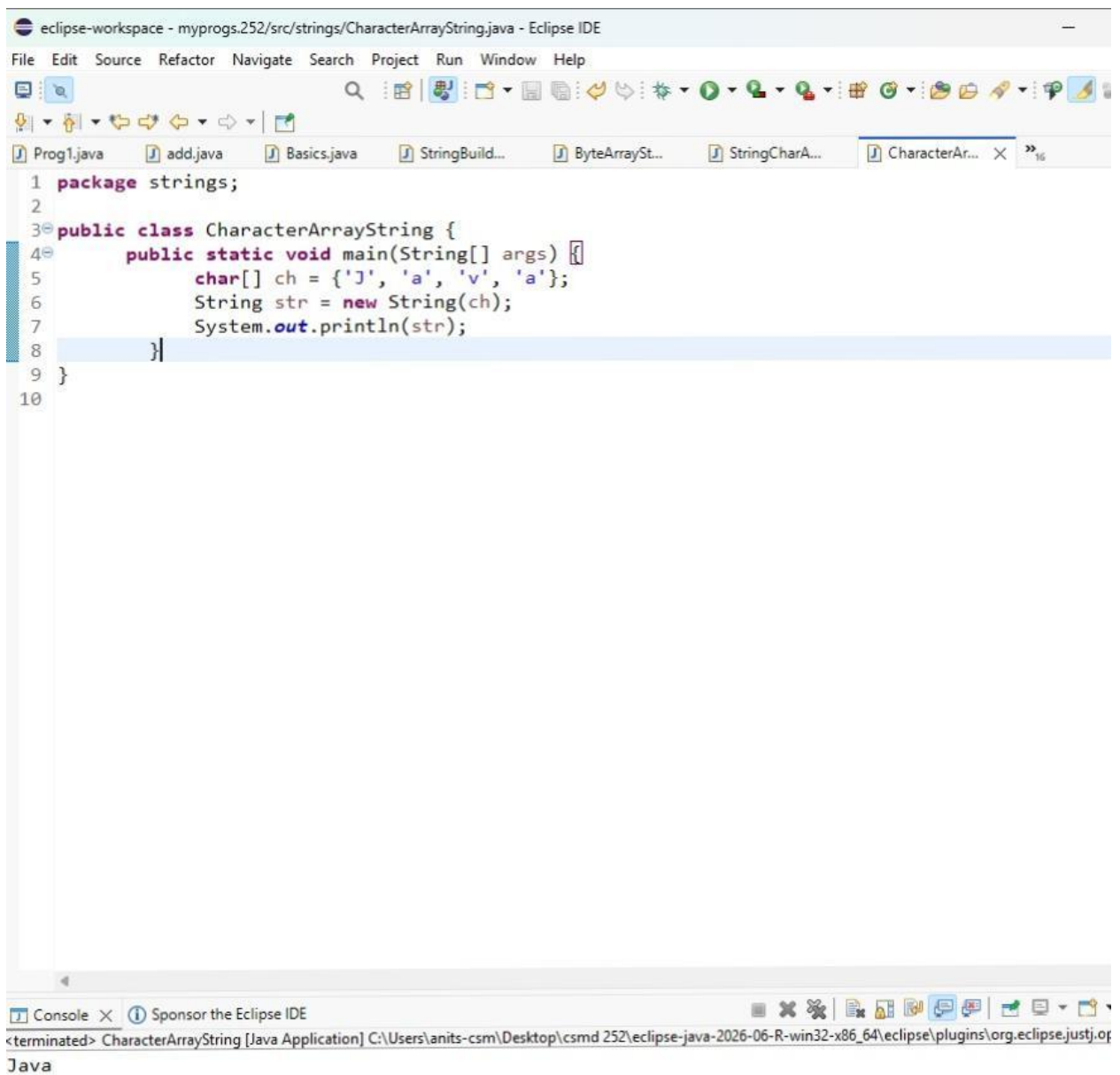
System.out.println(str);

}

}
```

Output

Java



The screenshot shows the Eclipse IDE interface. The top toolbar includes icons for File, Edit, Source, Refactor, Navigate, Search, Project, Run, Window, and Help. Below the toolbar is a package explorer showing a project named 'myprogs.252' with a package 'strings' containing several Java files. The editor displays the source code of 'CharacterArrayString.java'.

```
1 package strings;
2
3 public class CharacterArrayString {
4     public static void main(String[] args) {
5         char[] ch = {'J', 'a', 'v', 'a'};
6         String str = new String(ch);
7         System.out.println(str);
8     }
9 }
10
```

The bottom of the screenshot shows the console window with the output of the program:

```
<terminated> CharacterArrayString [Java Application] C:\Users\anits-csm\Desktop\csmd 252\eclipse-java-2026-06-R-win32-x86_64\eclipse\plugins\org.eclipse.justj.of
Java
```

5. String concat()

Problem Statement

Write a Java program to demonstrate the concat() method for joining two Strings.

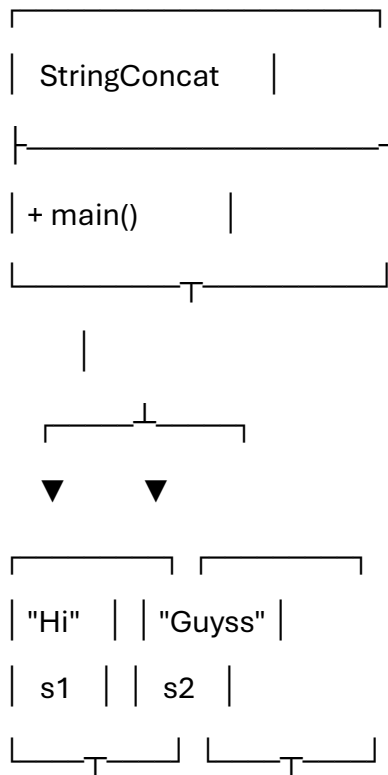
Steps

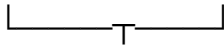
- Create two Strings "Hi" and "Guyss".
- Use the concat() method to join them.
- Display the result.

Algorithm

- Start.
- Create String s1 with "Hi".
- Create String s2 with "Guyss".
- Concatenate s2 with s1.
- Display the result.
- Stop.

UML Diagram





concat()



"HiGuyss"

Code

```
package strings;

class StringConcat {

    public static void main(String[] args) {

        String s1 = "Hi";

        String s2 = "Guyss";

        System.out.println("Result = " + s1.concat(s2));

    }

}
```

Output

Result = HiGuyss

The screenshot shows the Eclipse IDE interface. The top toolbar includes icons for File, Edit, Source, Refactor, Navigate, Search, Project, Run, Window, and Help. Below the toolbar is a package explorer showing a project named 'myprogs.252' with a sub-package 'strings'. The main editor displays the file 'StringConcat.java' with the following code:

```
1 package strings;
2
3 public class StringConcat {
4     public static void main(String[] args) {
5         String s1 = "Hi";
6         String s2 = "Guyss";
7         System.out.println("Result = " + s1.concat(s2));
8     }
9 }
10
```

The bottom of the IDE features a console window with the output: 'Result = HiGuyss'.

6. String isEmpty()

Problem Statement

Write a Java program to demonstrate the isEmpty() method to check whether a String contains no characters.

Steps

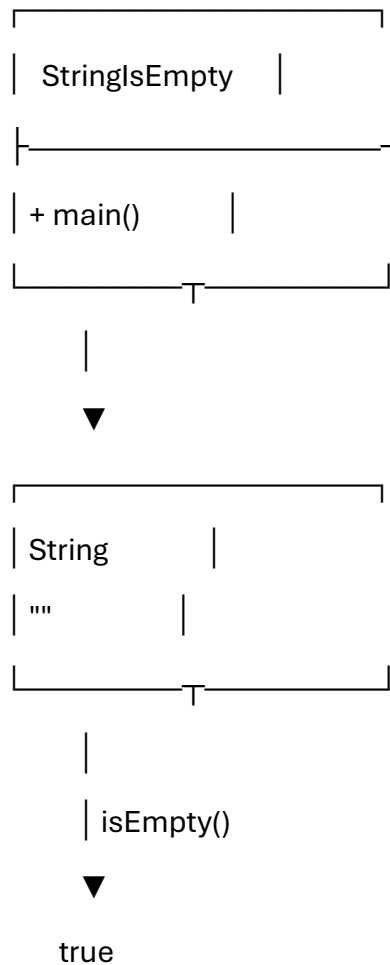
- Create an empty String.
- Use the isEmpty() method.
- Display the result.

Algorithm

- Start.

- Create an empty String.
- Call the isEmpty() method.
- Display the Boolean result.
- Stop.

UML Diagram



Code

```

package strings;

class StringIsEmpty {

    public static void main(String[] args) {

        String s = "";

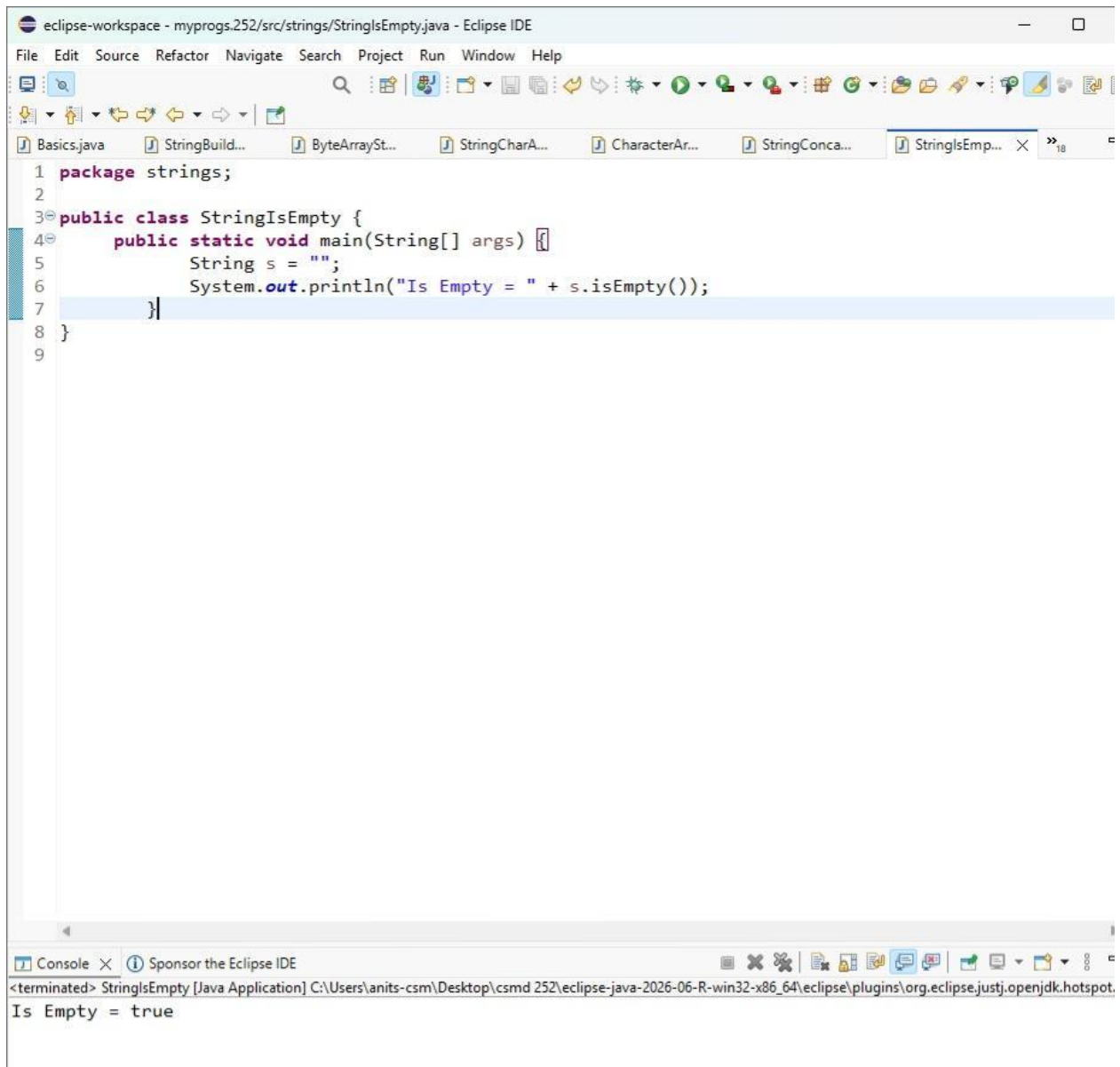
        System.out.println("Is Empty = " + s.isEmpty());

    }
  
```

```
}
```

Output

Is Empty = true



The screenshot shows the Eclipse IDE interface. The main editor window displays the following Java code:

```
1 package strings;
2
3 public class StringIsEmpty {
4     public static void main(String[] args) {
5         String s = "";
6         System.out.println("Is Empty = " + s.isEmpty());
7     }
8 }
9
```

The console window at the bottom shows the output of the program:

```
<terminated> StringIsEmpty [Java Application] C:\Users\anits-csm\Desktop\csmd 252\eclipse-java-2026-06-R-win32-x86_64\eclipse\plugins\org.eclipse.justj.openjdk.hotspot.
Is Empty = true
```

7. String equals()

Problem Statement

Write a Java program to demonstrate the equals() method for comparing two Strings.

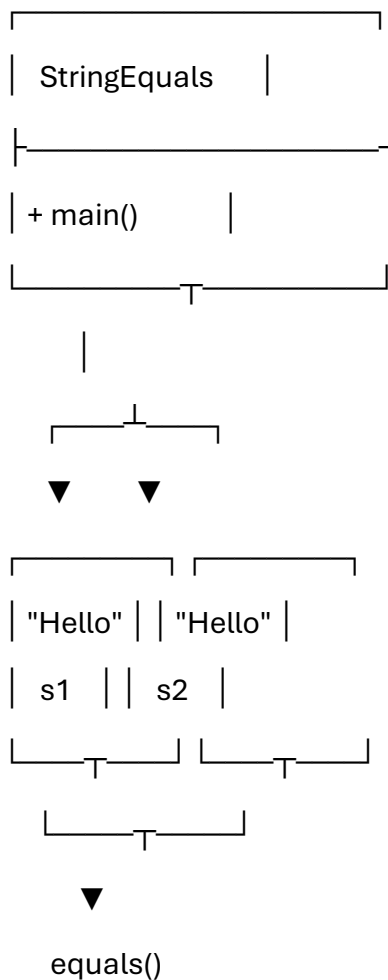
Steps

- Create two Strings with the same value.
- Compare them using equals().
- Display the result.

Algorithm

- Start.
- Create String s1 with "Hello".
- Create String s2 with "Hello".
- Compare both Strings using equals().
- Display the result.
- Stop.

UML Diagram



|



true

Code

```
package strings;

class StringEquals {

    public static void main(String[] args) {

        String s1 = "Hello";

        String s2 = "Hello";

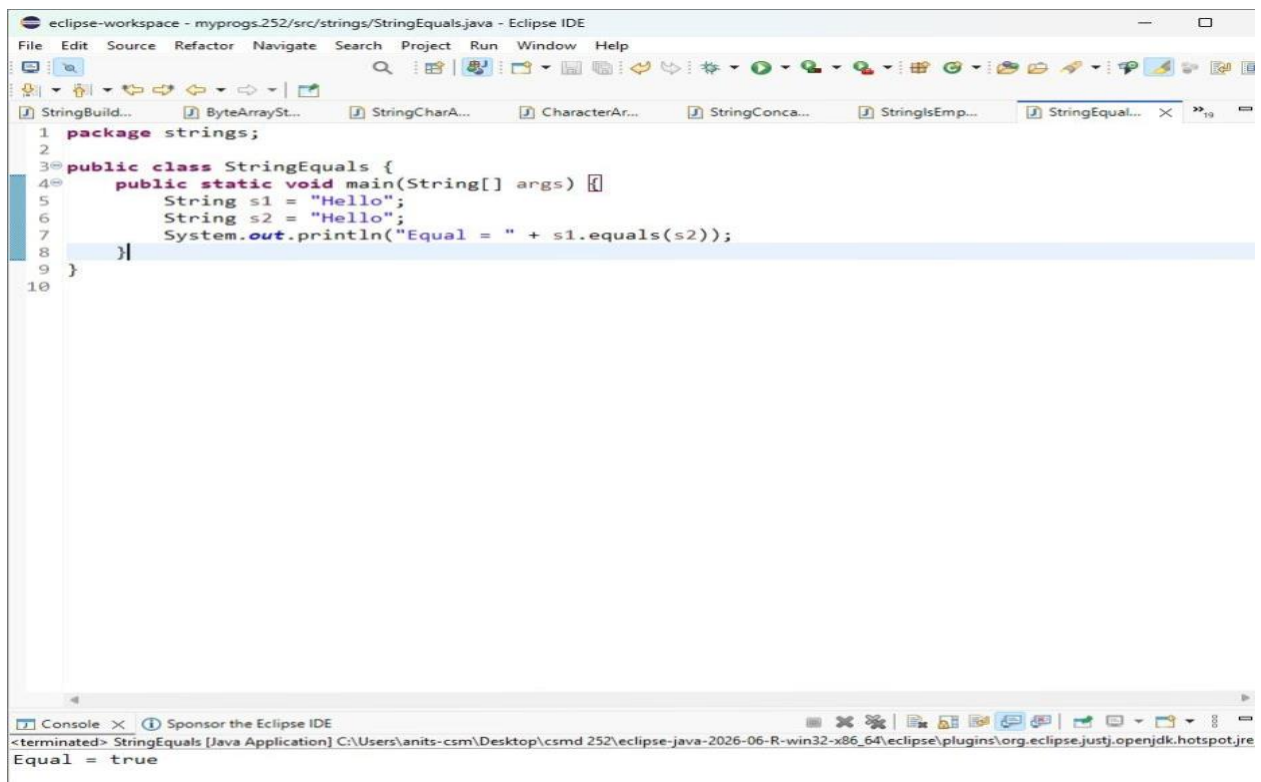
        System.out.println("Equal = " + s1.equals(s2));

    }

}
```

Output

Equal = true



8. Formatted String

Problem Statement

Write a Java program to demonstrate the `String.format()` method for creating a formatted String.

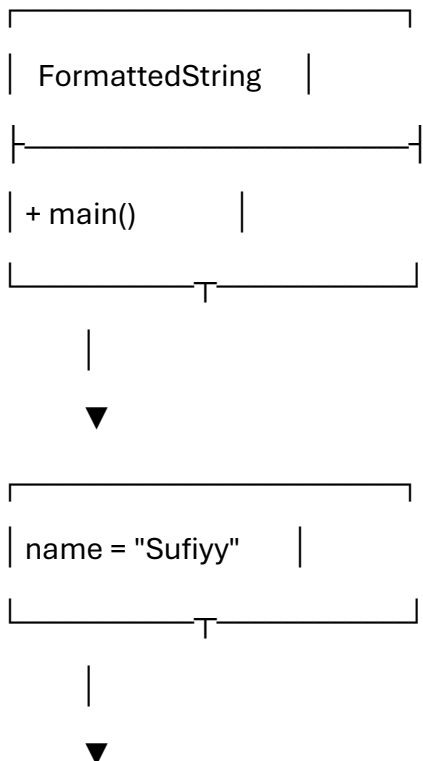
Steps

- Create a String variable containing a name.
- Use `String.format()` with `%s`.
- Display the formatted String.

Algorithm

- Start.
- Create a String variable name.
- Use `String.format()` to insert the name into the message.
- Display the formatted String.
- Stop.

UML Diagram



String.format()

|



```
| "Hello Sufiyy" |
```

Code

```
package strings;

class FormattedString {

    public static void main(String[] args) {

        String name = "Sufiyy";

        String str = String.format("Hello %s", name);

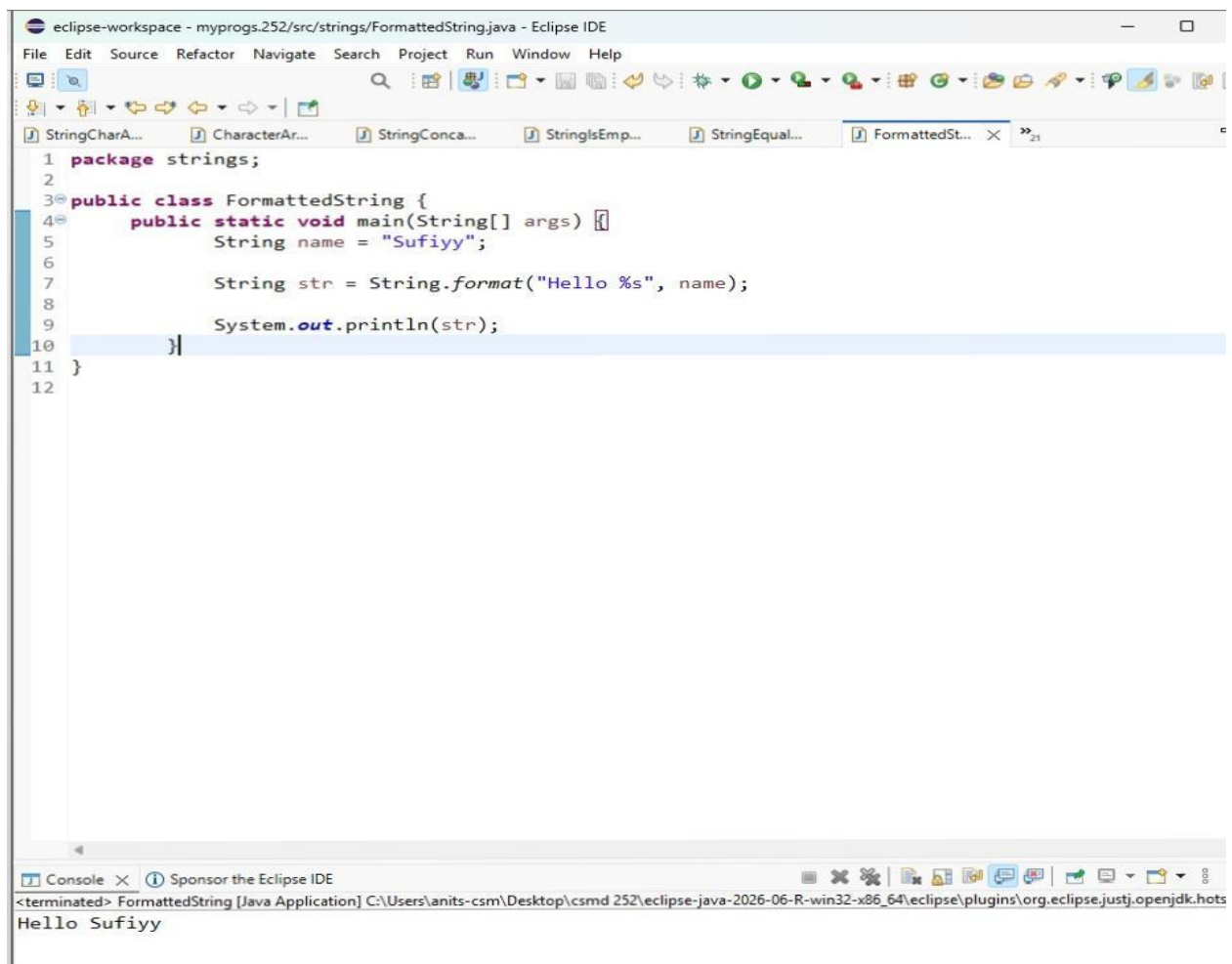
        System.out.println(str);

    }

}
```

Output

Hello Sufiyy



The screenshot shows the Eclipse IDE interface. The title bar reads "eclipse-workspace - myprogs.252/src/strings/FormattedString.java - Eclipse IDE". The menu bar includes File, Edit, Source, Refactor, Navigate, Search, Project, Run, Window, and Help. The toolbar contains various icons for file operations and development tools. The Package Explorer on the left shows a project named "strings" with several files: StringCharA..., CharacterAr..., StringConca..., StringIsEmp..., StringEqual..., and FormattedSt... The main editor displays the following Java code:

```
1 package strings;
2
3 public class FormattedString {
4     public static void main(String[] args) {
5         String name = "Sufiyy";
6
7         String str = String.format("Hello %s", name);
8
9         System.out.println(str);
10    }
11 }
12
```

The Console window at the bottom shows the output: "Hello Sufiyy". The console title is "Console" and it includes a link to "Sponsor the Eclipse IDE". The console output is: "<terminated> FormattedString [Java Application] C:\Users\anits-csm\Desktop\csmd 252\eclipse-java-2026-06-R-win32-x86_64\eclipse\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64.jre\bin\java.exe -Xmx256m -Djava.awt.headless=true -Djava.class.path=C:\Users\anits-csm\Desktop\csmd 252\src\strings\FormattedString.java -Djava.library.path=C:\Users\anits-csm\Desktop\csmd 252\src\strings\lib Hello Sufiyy".

9. String indexOf()

Problem Statement

Write a Java program to demonstrate the indexOf() method to find the position of a character in a String.

Steps

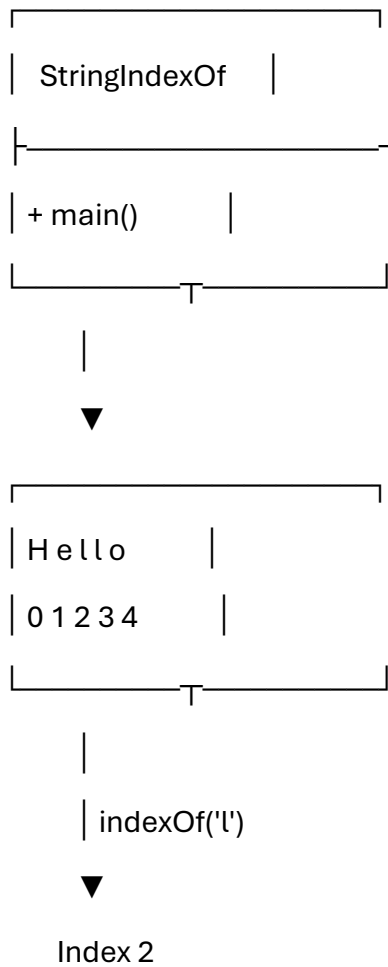
- Create the String "Hello".
- Search for the character 'l'.
- Display its index.

Algorithm

- Start.

- Create a String "Hello".
- Call indexOf('l').
- Display the returned index.
- Stop.

UML Diagram



Code

```

package strings;

class StringIndexOf {

    public static void main(String[] args) {

        String s = "Hello";

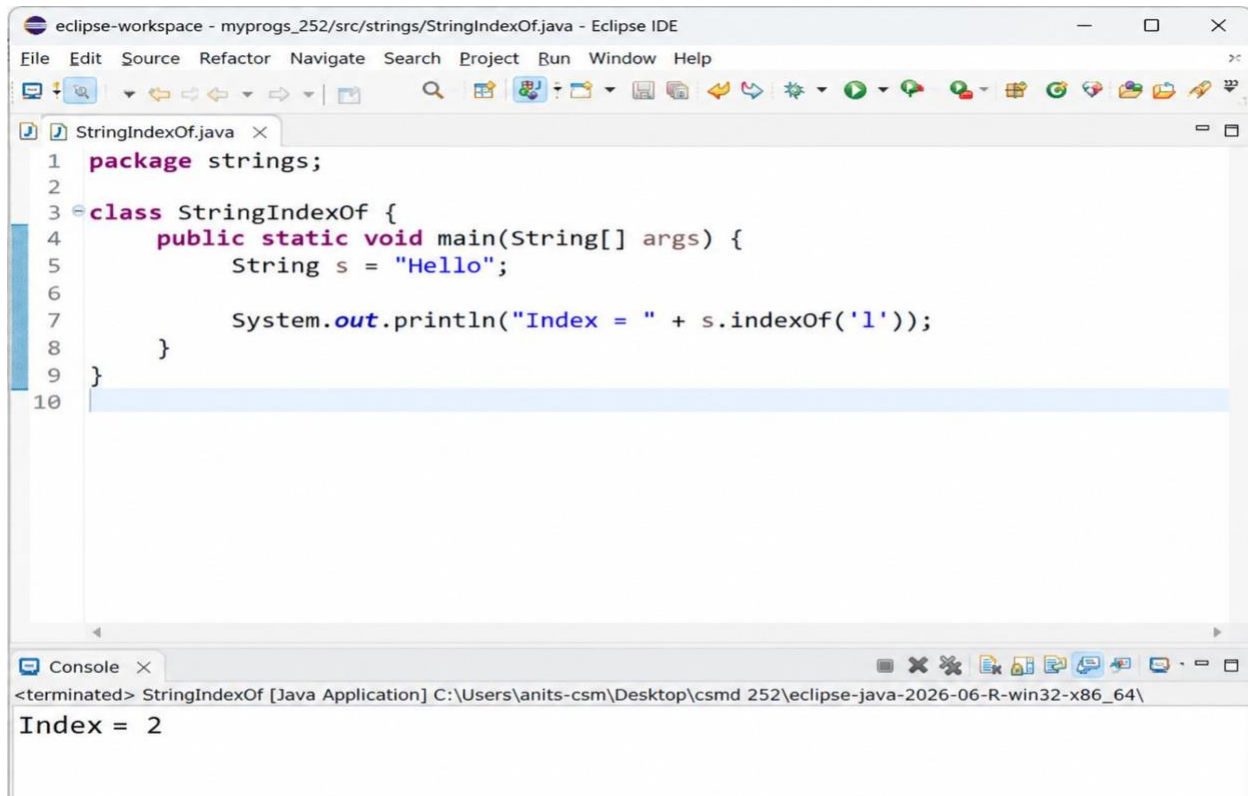
        System.out.println("Index = " + s.indexOf('l'));

    }
  
```

```
}
```

Output

Index = 2



The screenshot shows the Eclipse IDE interface. The top part is the 'StringIndexOf.java' editor, which contains the following code:

```
1 package strings;
2
3 class StringIndexOf {
4     public static void main(String[] args) {
5         String s = "Hello";
6
7         System.out.println("Index = " + s.indexOf('l'));
8     }
9 }
10
```

The bottom part is the 'Console' window, which shows the output of the program:

```
<terminated> StringIndexOf [Java Application] C:\Users\anits-csm\Desktop\csmd 252\eclipse-java-2026-06-R-win32-x86_64\
Index = 2
```

10. Convert String to Lowercase

Problem Statement

Write a Java program to demonstrate the `toLowerCase()` method for converting all characters of a String into lowercase.

Steps

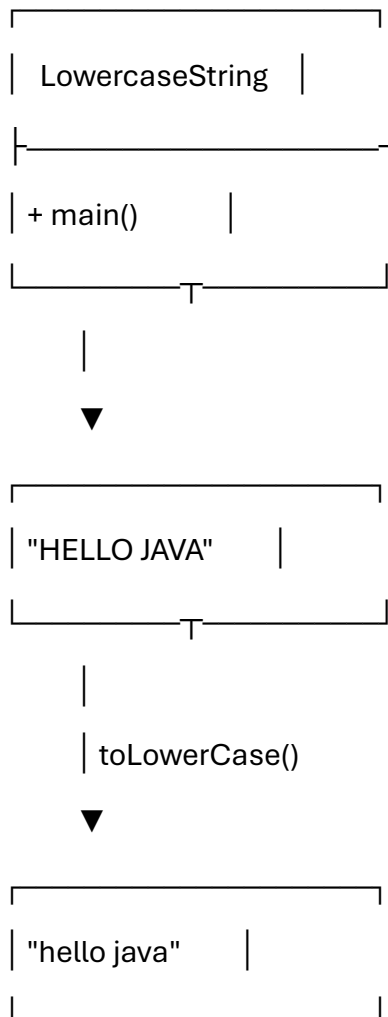
- Create a String containing uppercase letters.
- Call the `toLowerCase()` method.
- Display the converted String.

Algorithm

- Start.

- Create String "HELLO JAVA".
- Call toLowerCase().
- Display the lowercase String.
- Stop.

UML Diagram



Code

```
package strings;

public class LowercaseString {

    public static void main(String[] args) {

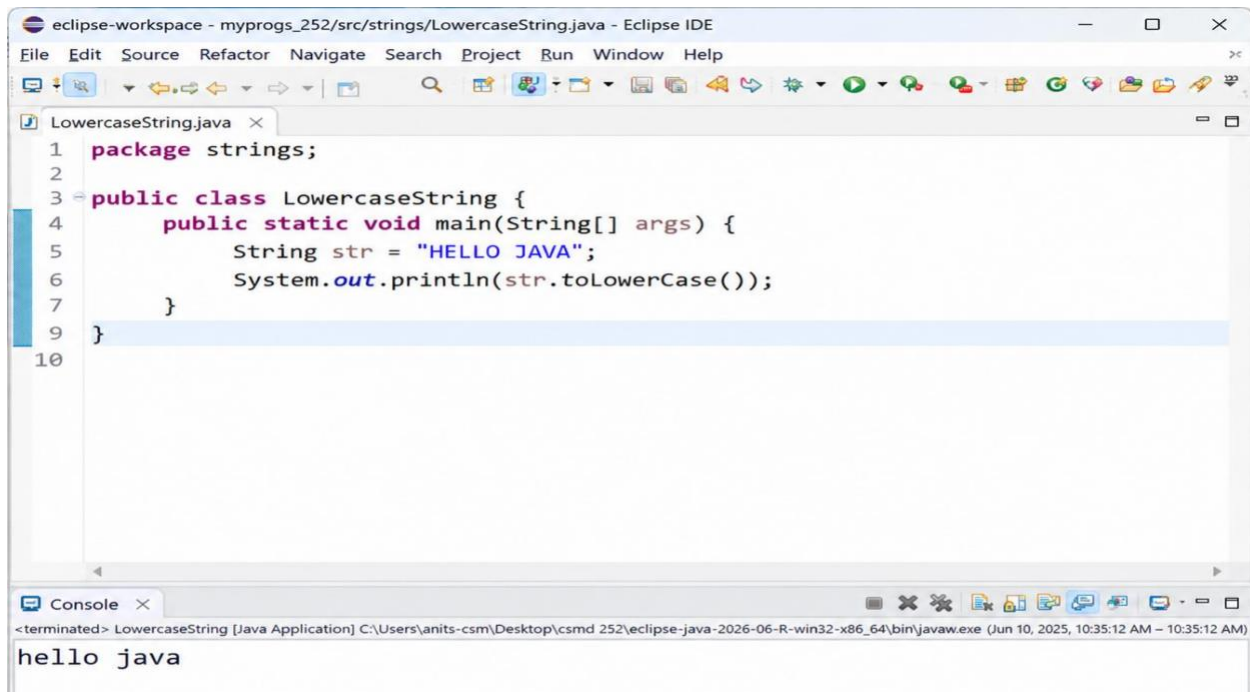
        String str = "HELLO JAVA";

        System.out.println(str.toLowerCase());
```

```
}  
  
}
```

Output

hello java



The screenshot shows the Eclipse IDE interface. The top toolbar includes menus like File, Edit, Source, Refactor, Navigate, Search, Project, Run, Window, and Help. Below the toolbar, the 'LowercaseString.java' file is open, displaying the following code:

```
1 package strings;  
2  
3 public class LowercaseString {  
4     public static void main(String[] args) {  
5         String str = "HELLO JAVA";  
6         System.out.println(str.toLowerCase());  
7     }  
8 }  
9  
10
```

At the bottom, the 'Console' window shows the output of the program:

```
<terminated> LowercaseString [Java Application] C:\Users\anits-csm\Desktop\csmd 252\eclipse-java-2026-06-R-win32-x86_64\bin\javaw.exe (Jun 10, 2025, 10:35:12 AM - 10:35:12 AM)  
hello java
```

11. Multiline String

Problem Statement

Write a Java program to demonstrate a multiline String using Java text blocks.

Steps

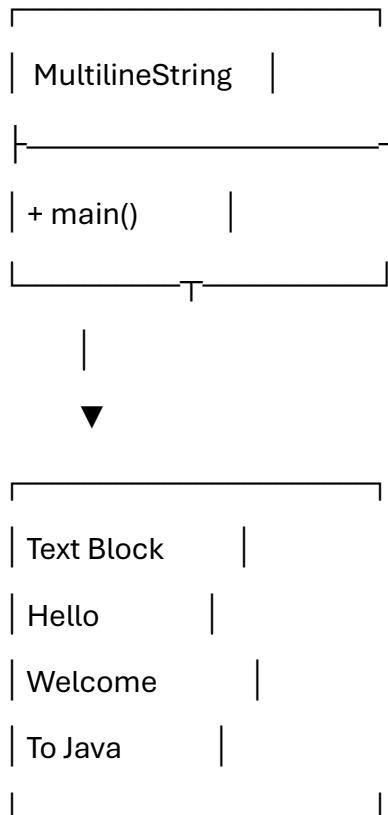
- Create a multiline String using triple double quotes.
- Store three lines of text.
- Display the String.

Algorithm

- Start.
- Create a multiline String.

- Store "Hello", "Welcome" and "To Java".
- Display the String.
- Stop.

UML Diagram



Code

```

package strings;

class MultilineString {

    public static void main(String[] args) {

        String str = ""

        Hello

        Welcome

        To Java

        """;
  
```

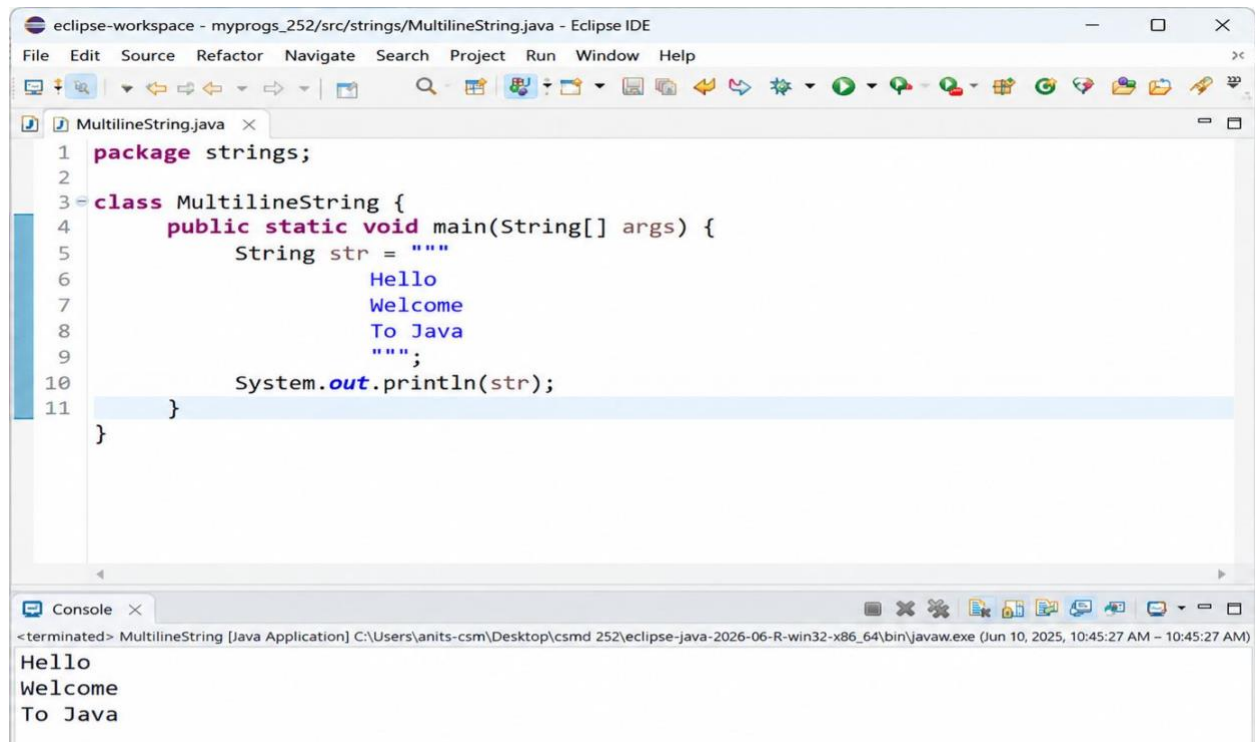
```
        System.out.println(str);
    }
}
```

Output

Hello

Welcome

To Java



The screenshot shows the Eclipse IDE interface. The main editor window displays the file `MultilineString.java` with the following code:

```
1 package strings;
2
3 class MultilineString {
4     public static void main(String[] args) {
5         String str = ""
6             Hello
7             Welcome
8             To Java
9             "";
10        System.out.println(str);
11    }
}
```

The console window at the bottom shows the output of the program:

```
<terminated> MultilineString [Java Application] C:\Users\anits-csm\Desktop\csmd 252\eclipse-java-2026-06-R-win32-x86_64\bin\javaw.exe (Jun 10, 2025, 10:45:27 AM - 10:45:27 AM)
Hello
Welcome
To Java
```

12. String Lowercase

Problem Statement

Write a Java program to demonstrate the `toLowerCase()` method to convert an uppercase String into lowercase.

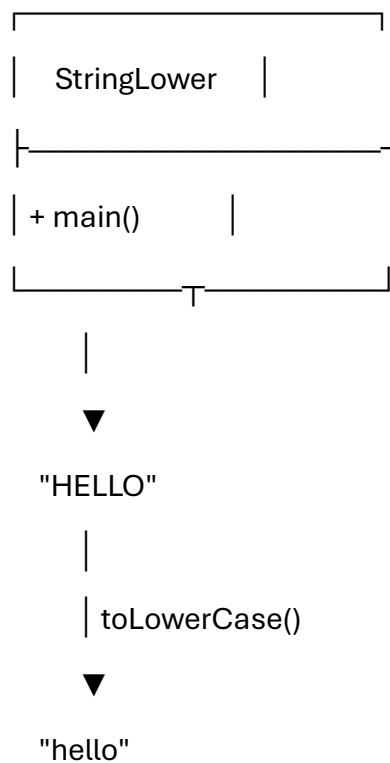
Steps

- Create the String "HELLO".
- Apply toLowerCase().
- Display the result.

Algorithm

- Start.
- Create String "HELLO".
- Call toLowerCase().
- Display the lowercase String.
- Stop.

UML Diagram



Code

```

package strings;

class StringLower {

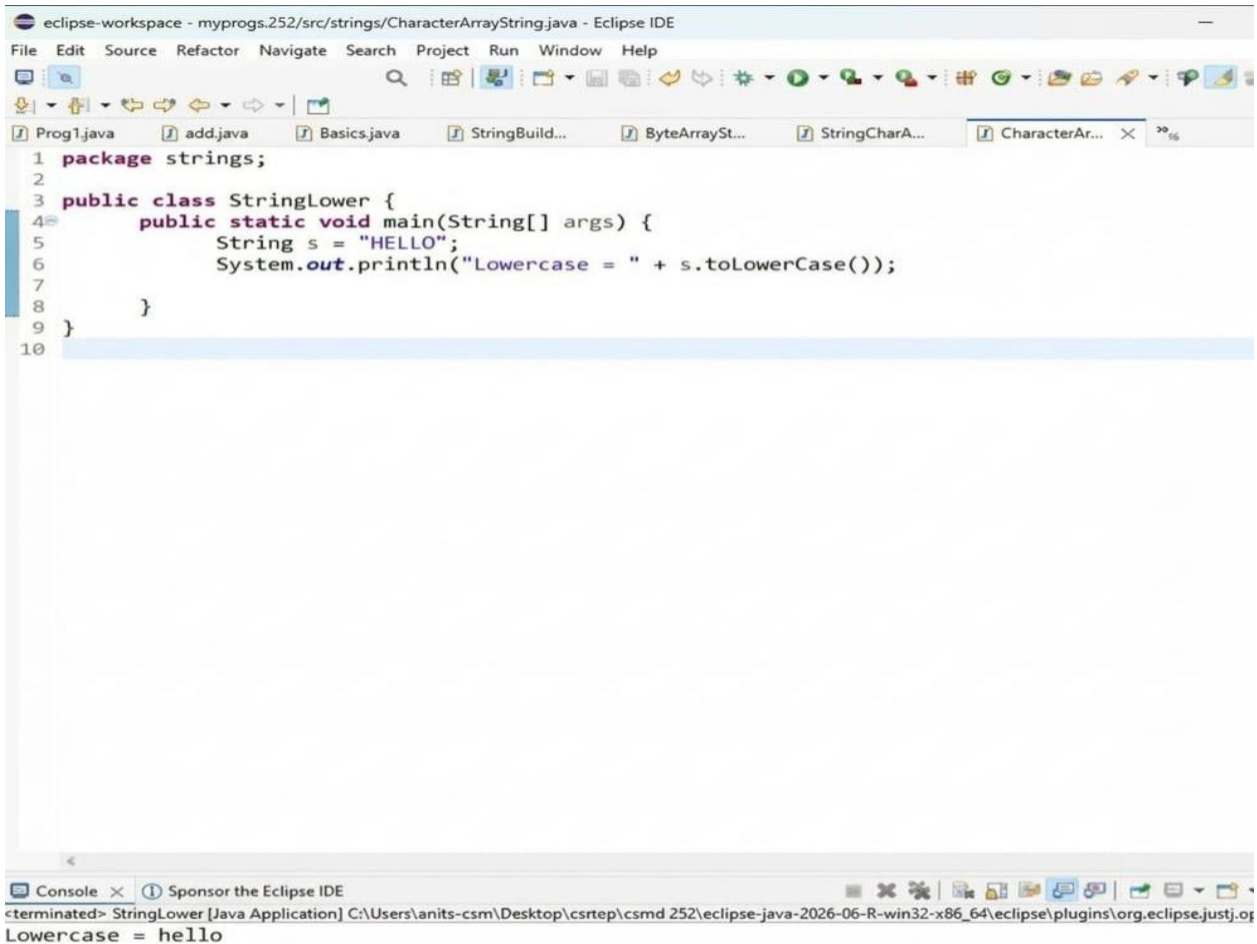
    public static void main(String[] args) {

        String s = "HELLO";
    }
}
  
```

```
        System.out.println("Lowercase = " + s.toLowerCase());
    }
}
```

Output

Lowercase = hello



The screenshot shows the Eclipse IDE interface. The main editor window displays a Java file named `CharacterArrayString.java` with the following code:

```
1 package strings;
2
3 public class StringLower {
4     public static void main(String[] args) {
5         String s = "HELLO";
6         System.out.println("Lowercase = " + s.toLowerCase());
7     }
8 }
9
10
```

The bottom of the IDE shows a console window with the output:

```
<terminated> StringLower [Java Application] C:\Users\anits-csm\Desktop\csrtcp\csmd 252\eclipse-java-2026-06-R-win32-x86_64\eclipse\plugins\org.eclipse.justj.o
Lowercase = hello
```

13. Special Character String

Problem Statement

Write a Java program to demonstrate a String containing special characters, numbers and letters.

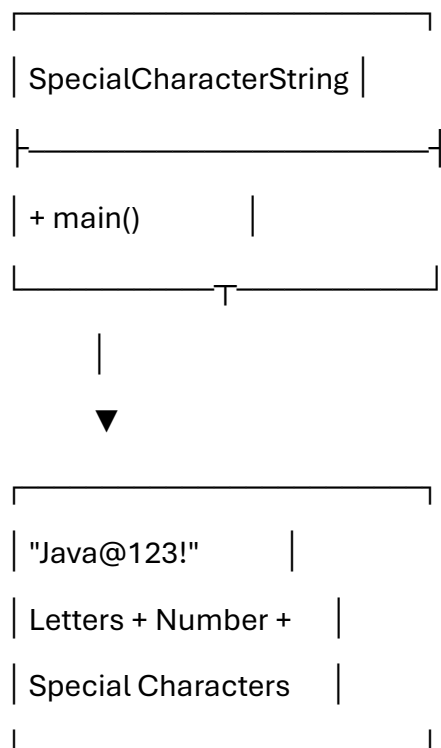
Steps

- Create a String containing "Java@123!".
- Display the String.

Algorithm

- Start.
- Create a String containing letters, a special character and numbers.
- Display the String.
- Stop.

UML Diagram



Code

```

package strings;

class SpecialCharacterString {

    public static void main(String[] args) {

        String str = "Java@123!";

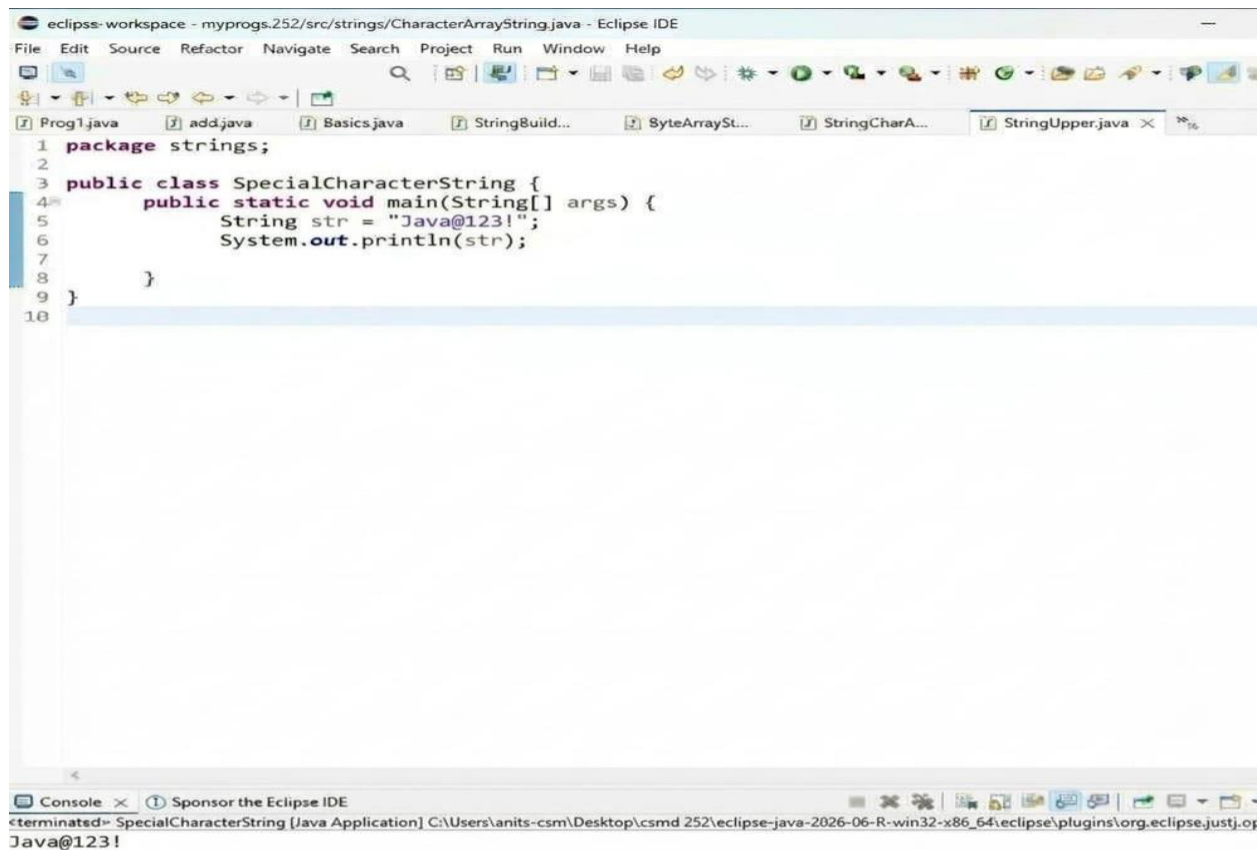
        System.out.println(str);

    }
  
```

```
}
```

Output

Java@123!



The screenshot shows the Eclipse IDE interface. The main editor window displays a Java file named `CharacterArrayString.java` with the following code:

```
1 package strings;
2
3 public class SpecialCharacterString {
4     public static void main(String[] args) {
5         String str = "Java@123!";
6         System.out.println(str);
7     }
8 }
9
10
```

The console window at the bottom shows the output of the program:

```
<terminated> SpecialCharacterString [Java Application] C:\Users\anits-csm\Desktop\csmd 252\eclipse-java-2026-06-R-win32-x86_64\eclipse\plugins\org.eclipse.justj.of
Java@123!
```

14. String Uppercase

Problem Statement

Write a Java program to demonstrate the `toUpperCase()` method to convert a lowercase String into uppercase.

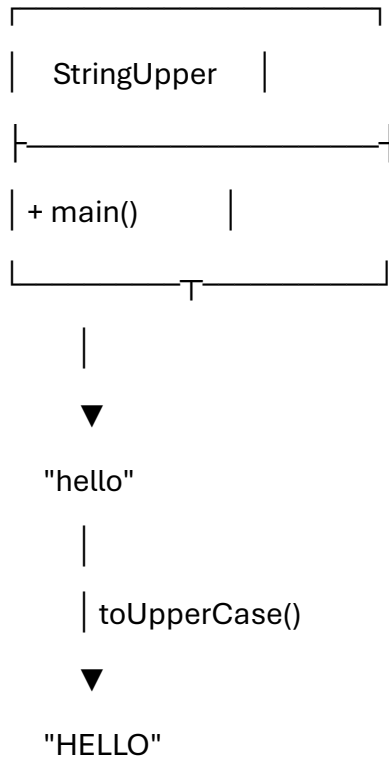
Steps

- Create the String "hello".
- Apply `toUpperCase()`.
- Display the result.

Algorithm

- Start.
- Create String "hello".
- Call toUpperCase().
- Display the uppercase String.
- Stop.

UML Diagram



Code

```

package strings;

class StringUpper {

    public static void main(String[] args) {

        String s = "hello";

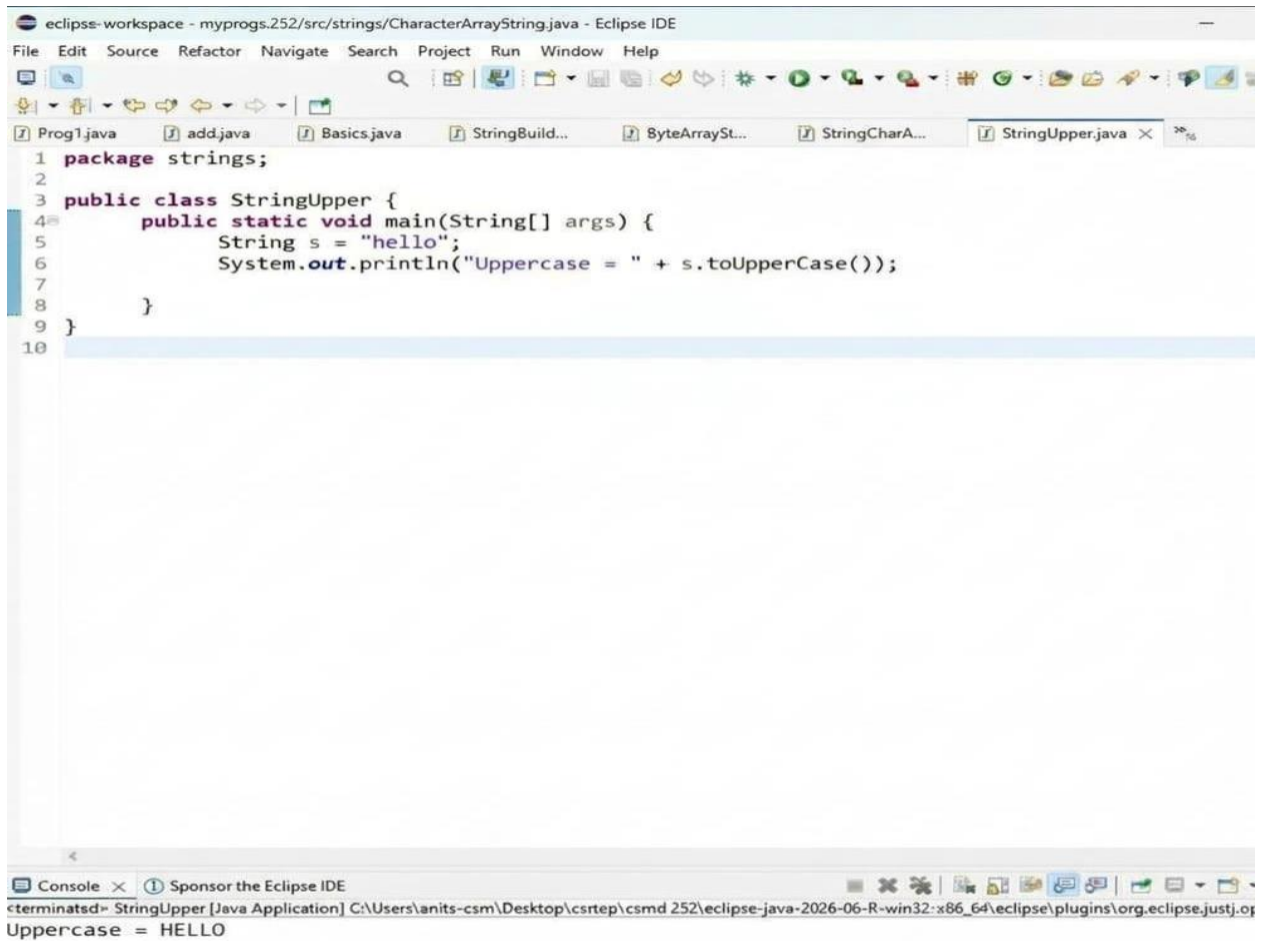
        System.out.println("Uppercase = " + s.toUpperCase());

    }

}
  
```

Output

Uppercase = HELLO



The screenshot shows the Eclipse IDE interface. The main editor window displays a Java file named `StringUpper.java` with the following code:

```
1 package strings;
2
3 public class StringUpper {
4     public static void main(String[] args) {
5         String s = "hello";
6         System.out.println("Uppercase = " + s.toUpperCase());
7     }
8 }
9
10
```

The console window at the bottom shows the output of the program:

```
<terminated> StringUpper [Java Application] C:\Users\anits-csm\Desktop\csrtcp\csmd 252\ eclipse-java-2026-06-R-win32-x86_64\ eclipse\plugins\org.eclipse.justj.oj
Uppercase = HELLO
```

15. String with Numbers

Problem Statement

Write a Java program to demonstrate how a number can be combined with a String using the + operator.

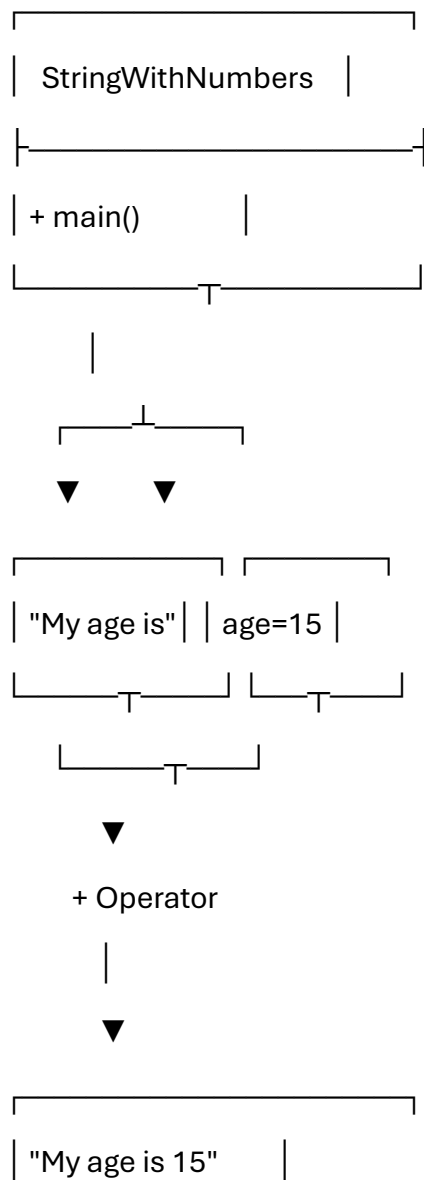
Steps

- Create an integer variable age.
- Assign the value 15.
- Combine the number with a String.
- Display the resulting String.

Algorithm

- Start.
- Declare an integer variable age.
- Assign 15 to age.
- Combine "My age is " with age.
- Store the result in a String.
- Display the String.
- Stop.

UML Diagram



Code

```
package strings;

class StringWithNumbers {

    public static void main(String[] args) {

        int age = 15;

        String str = "My age is " + age;

        System.out.println(str);

    }

}
```

Output

My age is 15

